

Universidade Federal de Mato Grosso – Campus Cuiabá



Ciência da Computação Bacharelado

Instituto de Computação

Docente:

Josiel Maimoni de Figueiredo

Discente:

Gabriel Martins Gomes

Projeto de Banco de Dados

Sistema de Gerenciamento do Banco Mundial de Sementes de Svalbard

Cuiabá – MT

2026

SUMÁRIO

1. PROJETO CONCEITUAL

1.1. ESPECIFICAÇÃO DE REQUISITOS

1.2. MODELAGEM DER & UML

2. PROJETO LÓGICO

2.1. MAPEAMENTO PARA RELACIONAL

2.2. INSERÇÃO DE DADOS

2.3. RELATÓRIOS DE CONSULTA

2.4. TRIGGERS

2.5. TIPOS NÃO CONVENCIONAIS

3. PROJETO FÍSICO

3.1. USO DE ÍNDICES

3.2. USO DE TABLESPACES

3.3. OTIMIZAÇÃO DOS RELATÓRIOS DE CONSULTA

3.4. CRIAÇÃO DE USUÁRIOS E PRIVILÉGIOS

3.5. DEFINIÇÃO DE RLS PARA USUÁRIOS

3.6. COMANDO PARA BACKUP LÓGICO

3.7. COMANDOS PARA BACKUP FÍSICO

4. MODELO NOSQL

1. PROJETO CONCEITUAL

1.1 Especificação de Requisitos

O projeto tem como objetivo a modelagem de um banco de dados relacional inspirado no Banco Mundial de Sementes de Svalbard (Svalbard Global Seed Vault), localizado no Ártico norueguês. O sistema foi projetado para gerenciar o ciclo de vida completo de amostras botânicas, desde a chegada à instalação até o armazenamento definitivo em câmaras frias a -18°C, abrangendo também o controle de acesso baseado em perfis institucionais, a classificação taxonômica das espécies e o rastreamento logístico dos contêineres físicos.

O sistema deverá gerenciar as instituições depositantes, que são as entidades internacionais — como a EMBRAPA (Brasil), o ICARDA (Síria/Líbano) e outros bancos nacionais de sementes — que enviam amostras e detêm os direitos legais sobre elas. Cada instituição é identificada por seu nome oficial e país de origem, e pode possuir um ou mais representantes científicos credenciados no sistema.

O controle de acesso é gerenciado por meio de uma hierarquia de usuários. O supertipo USUARIO armazena os dados comuns de autenticação de todos os operadores. A partir dele, dois subtipos são derivados: ADMINISTRADOR, que representa os funcionários locais da instalação em Svalbard e os auditores internacionais da ONU, com diferentes níveis de permissão e turnos de trabalho; e REPRESENTANTE, que são os cientistas delegados pelas instituições depositantes, obrigatoriamente vinculados a uma única instituição.

O inventário biológico é gerenciado pela entidade LOTE_SEMENTES, que é a entidade central do sistema. Cada lote registra a instituição proprietária, a classificação taxonômica da espécie (quando identificada), o contêiner físico onde está alocado (quando em armazenamento definitivo), a data de recebimento e seu status no ciclo de vida, que pode ser: Pendente, Aguardando Homologação, Em Quarentena, Pronto para Alocação, Armazenado, Inviável ou Retirado.

A classificação botânica é estruturada pela entidade CATEGORIA_TAXONOMICA, que utiliza um autorrelacionamento recursivo para representar a hierarquia filogenética completa (Família → Gênero → Espécie) em uma única tabela. Esse design garante a Terceira Forma Normal (3FN) e evita redundâncias ao armazenar a árvore taxonômica.

O armazenamento físico é gerenciado pelas entidades CAIXA e LOCALIZACAO. As caixas são os contêineres herméticos que agrupam os lotes nas câmaras frias, identificadas por QR Code ou código de barras, com capacidade máxima de 400 lotes. A localização mapeia o espaço tridimensional da instalação subterrânea por meio de sala/câmara, corredor, estante e prateleira. O histórico de movimentações das caixas entre localizações é registrado pela entidade HISTORICO_ALOCACAO_CAIXA.

Por fim, todas as transações críticas do sistema — recepção, homologação taxonômica, descarte sanitário, armazenamento final, mudança de caixa e retirada oficial — são registradas de forma imutável na tabela MOVIMENTACAO_AUDITORIA, que

armazena o lote afetado, o usuário responsável, o tipo da operação, o momento exato e uma assinatura digital hash para validação de integridade, em conformidade com as diretrizes internacionais de auditoria da ONU.

As três regras de negócio específicas do sistema são:

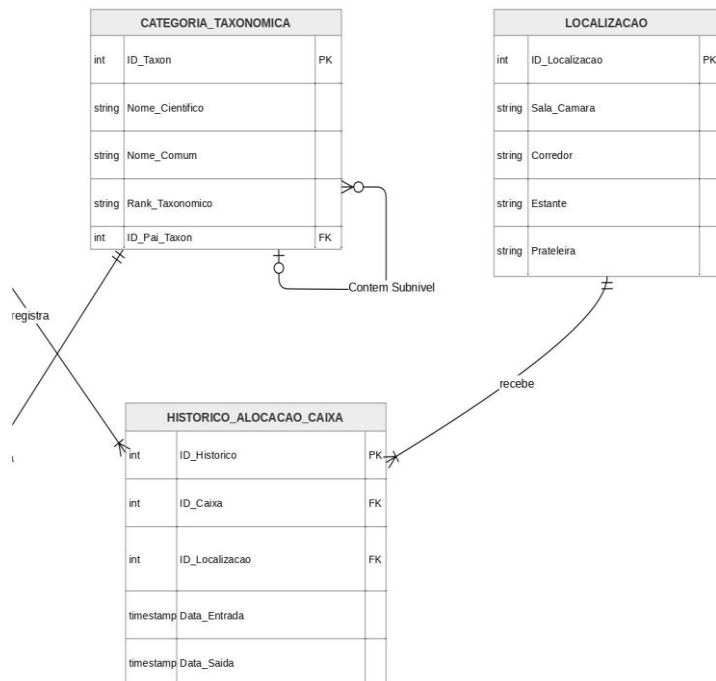
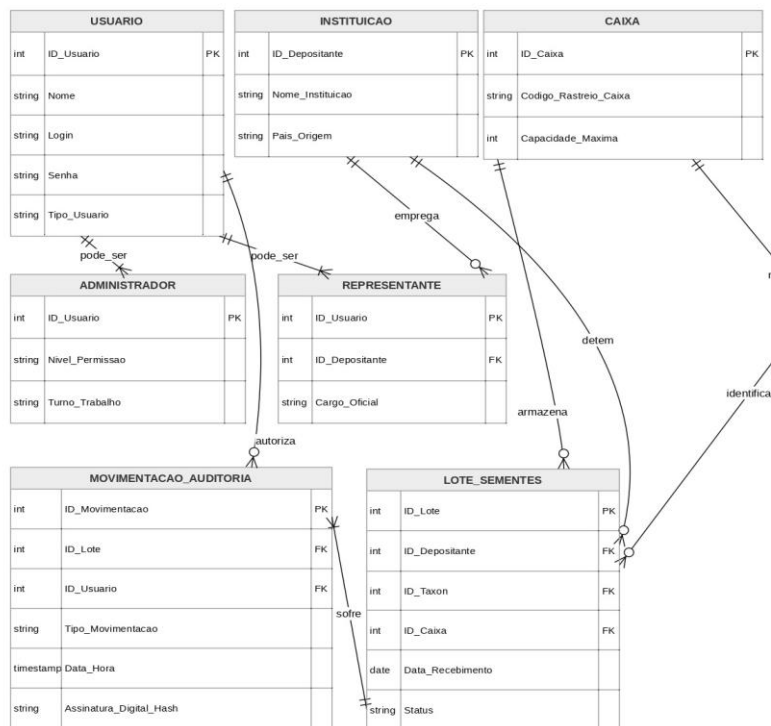
- **RN01** – Regra da "Caixa Preta" (Isolamento por País): Os direitos de propriedade intelectual das sementes pertencem estritamente à nação depositante. O banco de dados implementa Row-Level Security (RLS) nativa que, quando um usuário do tipo REPRESENTANTE realiza uma consulta na tabela LOTE_SEMENTES, injeta automaticamente um filtro restritivo para que ele visualize apenas os lotes de sua própria instituição.
- **RN02** – Controle de Capacidade Concorrente via Trigger: Uma caixa física não pode ultrapassar a capacidade máxima definida em Capacidade_Maxima. Um gatilho BEFORE INSERT OR UPDATE na tabela LOTE_SEMENTES verifica, em nível de isolamento serializável, se a inserção de um novo lote estouraria o limite; em caso afirmativo, aborta a transação com RAISE EXCEPTION e ROLLBACK.
- **RN03** – Imutabilidade dos Registros Históricos (Proibição de DELETE): Comandos DELETE são terminantemente bloqueados nas tabelas LOTE_SEMENTES, HISTORICO_ALOCACAO_CAIXA e MOVIMENTACAO_AUDITORIA por gatilhos BEFORE DELETE. Descartes ou retiradas são realizados via UPDATE no campo Status, o que dispara automaticamente um INSERT na tabela de auditoria.

1.2 Modelagem DER & UML

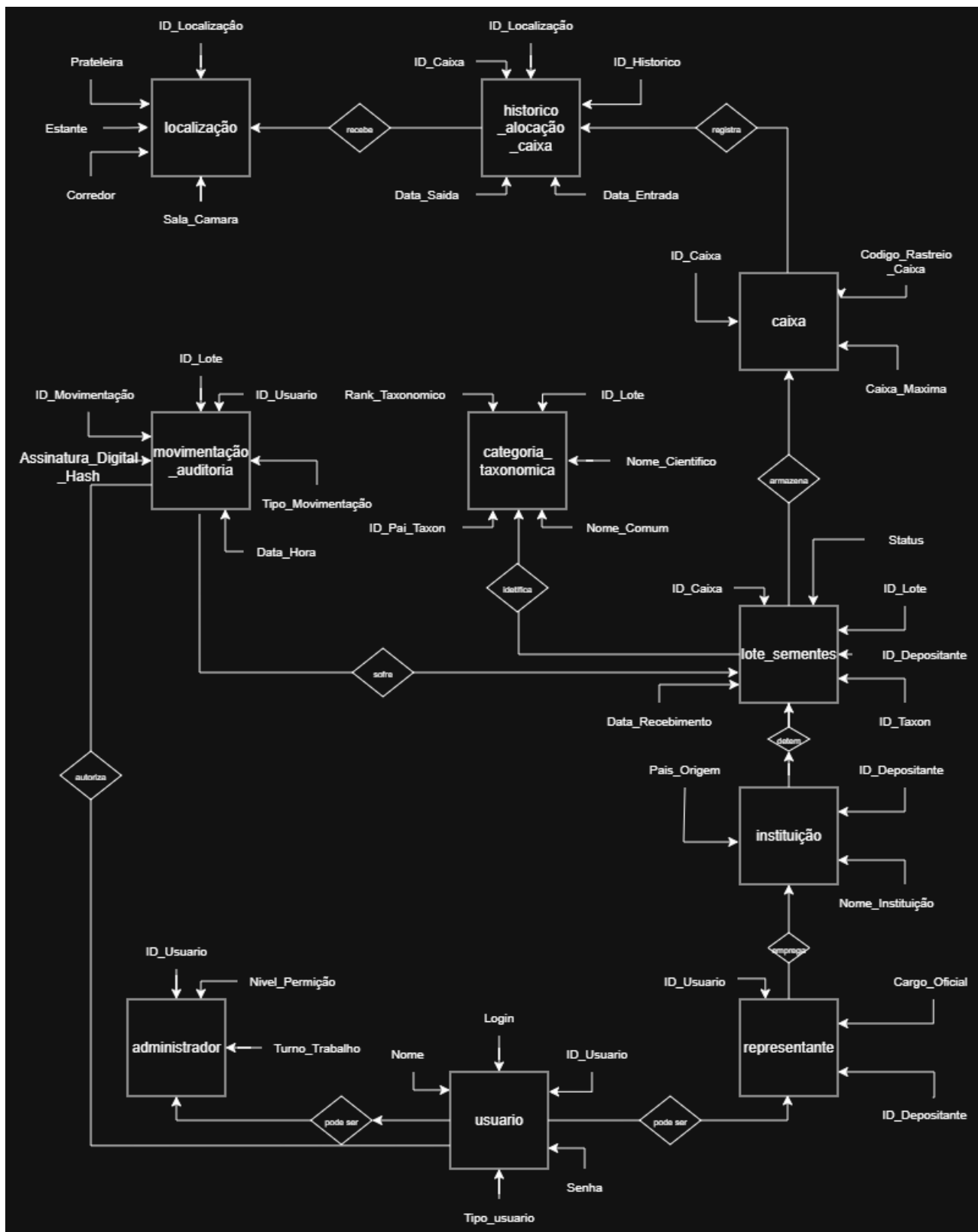
O Diagrama Entidade-Relacionamento (DER) do sistema Svalbard VaultDB é composto por 10 entidades organizadas em três camadas: entidades independentes (INSTITUICAO, USUARIO, CAIXA, LOCALIZACAO, CATEGORIA_TAXONOMICA), entidades dependentes/subtipos (ADMINISTRADOR, REPRESENTANTE, LOTE_SEMENTES) e entidades de junção e log (HISTORICO_ALOCACAO_CAIXA, MOVIMENTACAO_AUDITORIA).

Os principais relacionamentos são: USUARIO é especializado em ADMINISTRADOR ou REPRESENTANTE (1:1 exclusivo); INSTITUICAO emprega zero ou vários REPRESENTANTE (1:N); INSTITUICAO detém vários LOTE_SEMENTES (1:N); CATEGORIA_TAXONOMICA contém subnível de si mesma (1:N recursivo); CAIXA armazena vários LOTE_SEMENTES (1:N); CAIXA e LOCALIZACAO se relacionam via HISTORICO_ALOCACAO_CAIXA (N:M); LOTE_SEMENTES sofre múltiplas MOVIMENTACAO_AUDITORIA (1:N); e USUARIO autoriza múltiplas MOVIMENTACAO_AUDITORIA (1:N).

Modelagem UML



Modelagem DER



2. PROJETO LÓGICO

2.1 Mapeamento para Relacional

Tabelas Principais (Independentes)

INSTITUICAO

(ID_Depositante, Nome_Instituicao, Pais_Origem)

USUARIO

(ID_Usuario, Nome, Login, Senha, Tipo_Usuario)

CAIXA

(ID_Caixa, Codigo_Rastreio_Caixa, Capacidade_Maxima)

LOCALIZACAO

(ID_Localizacao, Sala_Camara, Corredor, Estante, Prateleira)

CATEGORIA_TAXONOMICA

(ID_Taxon, Nome_Cientifico, Nome_Comum, Rank_Taxonomico, ID_Pai_Taxon)

Tabelas Dependentes (Subtipos e Entidades com Relacionamentos)

ADMINISTRADOR

(ID_Usuario, Nivel_Permissao, Turno_Trabalho)

Chave Primária e Estrangeira: ID_Usuario referencia USUARIO(ID_Usuario)

REPRESENTANTE

(ID_Usuario, ID_Depositante, Cargo_Oficial)

Chaves Estrangeiras: ID_Usuario referencia USUARIO(ID_Usuario), ID_Depositante referencia INSTITUICAO(ID_Depositante)

LOTE_SEMENTES

(ID_Lote, ID_Depositante, ID_Taxon, ID_Caixa, Data_Recebimento, Status)

Chaves Estrangeiras: ID_Depositante referencia INSTITUICAO(ID_Depositante), ID_Taxon referencia CATEGORIA_TAXONOMICA(ID_Taxon), ID_Caixa referencia CAIXA(ID_Caixa)

Tabelas de Junção / Relacionamento N:M e Logs

HISTORICO_ALOCACAO_CAIXA

(ID_Historico, ID_Caixa, ID_Localizacao, Data_Entrada, Data_Saida)

Chaves Estrangeiras: ID_Caixa referencia CAIXA(ID_Caixa), ID_Localizacao referencia LOCALIZACAO(ID_Localizacao)

MOVIMENTACAO_AUDITORIA

(ID_Movimentacao, ID_Lote, ID_Usuario, Tipo_Movimentacao, Data_Hora, Assinatura_Digital_Hash)

Chaves Estrangeiras: ID_Lote referencia LOTE_SEMENTES(ID_Lote), ID_Usuario referencia USUARIO(ID_Usuario)

2.2 Inserção de Dados

Criação do Banco

```
-- =====
-- SVALBARD VAULTDB - CRIAÇÃO DAS TABELAS (PostgreSQL)
-- =====

-- Tabela: INSTITUICAO
CREATE TABLE INSTITUICAO (
    ID_Depositante SERIAL PRIMARY KEY,
    Nome_Instituicao VARCHAR(200) NOT NULL,
    Pais_Origem VARCHAR(100) NOT NULL
);

-- Tabela: USUARIO (supertipo)
CREATE TABLE USUARIO (
    ID_Usuario SERIAL PRIMARY KEY,
    Nome VARCHAR(150) NOT NULL,
    Login VARCHAR(100) NOT NULL UNIQUE,
    Senha VARCHAR(255) NOT NULL,
    Tipo_Usuario VARCHAR(20) NOT NULL,
    CONSTRAINT chk_tipo_usuario
        CHECK (Tipo_Usuario IN ('ADMINISTRADOR', 'REPRESENTANTE'))
);

-- Tabela: CAIXA
CREATE TABLE CAIXA (
    ID_Caixa SERIAL PRIMARY KEY,
    Codigo_Rastreio_Caixa VARCHAR(100) NOT NULL UNIQUE,
    Capacidade_Maxima INT NOT NULL DEFAULT 400
);

-- Tabela: LOCALIZACAO
CREATE TABLE LOCALIZACAO (
    ID_Localizacao SERIAL PRIMARY KEY,
    Sala_Camara VARCHAR(50) NOT NULL,
    Corredor VARCHAR(50) NOT NULL,
    Estante VARCHAR(50) NOT NULL,
    Prateleira VARCHAR(50) NOT NULL
);

-- Tabela: CATEGORIA_TAXONOMICA (autorrelacionamento recursivo)
CREATE TABLE CATEGORIA_TAXONOMICA (
    ID_Taxon SERIAL PRIMARY KEY,
    Nome_Cientifico VARCHAR(200) NOT NULL UNIQUE,
    Nome_Comum VARCHAR(200),
    Rank_Taxonomico VARCHAR(10) NOT NULL,
    ID_Pai_Taxon INT,
    CONSTRAINT chk_rank
        CHECK (Rank_Taxonomico IN ('FAMILIA', 'GENERO', 'ESPECIE')),
    CONSTRAINT fk_pai_taxon
        FOREIGN KEY (ID_Pai_Taxon)
        REFERENCES CATEGORIA_TAXONOMICA(ID_Taxon)
);

-- Tabela: ADMINISTRADOR (subtipo de USUARIO)
CREATE TABLE ADMINISTRADOR (
```

```

        ID_Usuario          INT          PRIMARY KEY,
        Nivel_Permissao    VARCHAR(50)   NOT NULL,
        Turno_Trabalho     VARCHAR(30)   NOT NULL,
        CONSTRAINT fk_admin_usuario
            FOREIGN KEY (ID_Usuario)
            REFERENCES USUARIO(ID_Usuario)
    );

-- Tabela: REPRESENTANTE (subtipo de USUARIO)
CREATE TABLE REPRESENTANTE (
    ID_Usuario          INT          PRIMARY KEY,
    ID_Depositante      INT          NOT NULL,
    Cargo_Oficial       VARCHAR(100) NOT NULL,
    CONSTRAINT fk_repr_usuario
        FOREIGN KEY (ID_Usuario)
        REFERENCES USUARIO(ID_Usuario),
    CONSTRAINT fk_repr_instituicao
        FOREIGN KEY (ID_Depositante)
        REFERENCES INSTITUICAO(ID_Depositante)
);

-- Tabela: LOTE_SEMENTES (entidade central)
CREATE TABLE LOTE_SEMENTES (
    ID_Lote              SERIAL        PRIMARY KEY,
    ID_Depositante       INT          NOT NULL,
    ID_Taxon             INT,
    ID_Caixa             INT,
    Data_Recebimento     DATE          NOT NULL,
    Status               VARCHAR(30) NOT NULL,
    CONSTRAINT chk_status CHECK (Status IN (
        'Pendente', 'Aguardando Homologacao', 'Em Quarentena',
        'Pronto para Alocacao', 'Armazenado', 'Inviavel', 'Retirado'
    )),
    CONSTRAINT fk_lote_depositante
        FOREIGN KEY (ID_Depositante)
        REFERENCES INSTITUICAO(ID_Depositante),
    CONSTRAINT fk_lote_taxon
        FOREIGN KEY (ID_Taxon)
        REFERENCES CATEGORIA_TAXONOMICA(ID_Taxon),
    CONSTRAINT fk_lote_caixa
        FOREIGN KEY (ID_Caixa)
        REFERENCES CAIXA(ID_Caixa)
);

-- Tabela: HISTORICO_ALOCACAO_CAIXA (N:M entre CAIXA e LOCALIZACAO)
CREATE TABLE HISTORICO_ALOCACAO_CAIXA (
    ID_Historico         SERIAL        PRIMARY KEY,
    ID_Caixa             INT          NOT NULL,
    ID_Localizacao        INT          NOT NULL,
    Data_Entrada          TIMESTAMP NOT NULL,
    Data_Saida           TIMESTAMP,
    CONSTRAINT fk_hist_caixa
        FOREIGN KEY (ID_Caixa)
        REFERENCES CAIXA(ID_Caixa),
    CONSTRAINT fk_hist_localizacao
        FOREIGN KEY (ID_Localizacao)
        REFERENCES LOCALIZACAO(ID_Localizacao)
);

```

```
-- Tabela: MOVIMENTACAO_AUDITORIA (log imutável)
CREATE TABLE MOVIMENTACAO_AUDITORIA (
    ID_Movimentacao      SERIAL          PRIMARY KEY,
    ID_Lote               INT             NOT NULL,
    ID_Usuario            INT             NOT NULL,
    Tipo_Movimentacao     VARCHAR(30)     NOT NULL,
    Data_Hora              TIMESTAMP      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    Assinatura_Digital_Hash VARCHAR(255) NOT NULL,
    CONSTRAINT chk_tipo_mov CHECK (Tipo_Movimentacao IN (
        'Rececao', 'Homologacao Taxonomica', 'Descarte Sanitario',
        'Armazenamento Final', 'Mudanca de Caixa', 'Retirada'
    )),
    CONSTRAINT fk_mov_lote
        FOREIGN KEY (ID_Lote)
        REFERENCES LOTE_SEMENTES(ID_Lote),
    CONSTRAINT fk_mov_usuario
        FOREIGN KEY (ID_Usuario)
        REFERENCES USUARIO(ID_Usuario)
);
```

Inserção dos Dados

```
-- Inserção de dados de exemplo

-- Instituições depositantes
INSERT INTO INSTITUICAO (Nome_Instituicao, Pais_Origem) VALUES
('EMBRAPA - Empresa Brasileira de Pesquisa Agropecuária', 'Brasil'),
('ICARDA - International Center for Agricultural Research', 'Líbano'),
('Nordic Genetic Resource Center (NordGen)', 'Noruega'),
('National Bureau of Plant Genetic Resources (NBPGR)', 'Índia'),
('USDA - Agricultural Research Service', 'Estados Unidos');

-- Usuários (Administradores e Representantes)
INSERT INTO USUARIO (Nome, Login, Senha, Tipo_Usuario) VALUES
('Erik Solberg', 'erik.solberg', '$2b$12$Kg8aHfXzUuq1', 'ADMINISTRADOR'),
('Anika Thorvaldsen', 'anika.thor', '$2b$12$Pq7dGhYrVvm2', 'ADMINISTRADOR'),
('Dr. Carlos Nunes', 'carlos.nunes', '$2b$12$Lr9eKiZsWWn3', 'REPRESENTANTE'),
('Dra. Priya Menon', 'priya.menon', '$2b$12$Ms0fLjAtXXo4', 'REPRESENTANTE'),
('Dr. James Carter', 'james.carter', '$2b$12$Nt1gMkBuYyp5', 'REPRESENTANTE');

INSERT INTO ADMINISTRADOR (ID_Usuario, Nivel_Permissao, Turno_Trabalho) VALUES
(1, 'Master', 'Diurno'),
(2, 'Auditor', 'Noturno');

INSERT INTO REPRESENTANTE (ID_Usuario, ID_Depositante, Cargo_Oficial) VALUES
(3, 1, 'Pesquisador Sênior em Recursos Genéticos'),
(4, 4, 'Curadora de Germoplasma'),
(5, 5, 'Diretor de Conservação Ex Situ');

-- Localizações nas câmaras
INSERT INTO LOCALIZACAO (Sala_Camara, Corredor, Estante, Prateleira) VALUES
('Câmara 1', 'A', 'E01', 'P1'),
('Câmara 1', 'A', 'E01', 'P2'),
('Câmara 1', 'B', 'E03', 'P1'),
('Câmara 2', 'A', 'E01', 'P1'),
```

```

('Câmara 2', 'B', 'E02', 'P3');

-- Caixas herméticas
INSERT INTO CAIXA (Codigo_Rastreio_Caixa, Capacidade_Maxima) VALUES
('SVL-BOX-2024-001', 400),
('SVL-BOX-2024-002', 400),
('SVL-BOX-2024-003', 400),
('SVL-BOX-2024-004', 400);

-- Histórico de alocação das caixas
INSERT INTO HISTORICO_ALOCACAO_CAIXA
(ID_Caixa, ID_Localizacao, Data_Entrada, Data_Saida) VALUES
(1, 1, '2024-01-10 08:00:00', NULL),
(2, 2, '2024-01-10 08:00:00', NULL),
(3, 4, '2024-02-15 09:30:00', NULL),
(4, 5, '2024-03-20 10:00:00', NULL);

-- Taxonomia (Família → Gênero → Espécie)
INSERT INTO CATEGORIA_TAXONOMICA
(Nome_Cientifico, Nome_Comum, Rank_Taxonomico, ID_Pai_Taxon) VALUES
('Poaceae', 'Gramíneas', 'FAMILIA', NULL),
('Triticum', 'Trigo', 'GENERO', 1),
('Oryza', 'Arroz', 'GENERO', 1),
('Zea', 'Milho', 'GENERO', 1),
('Triticum aestivum', 'Trigo Comum', 'ESPECIE', 2),
('Triticum durum', 'Trigo Durum', 'ESPECIE', 2),
('Oryza sativa', 'Arroz Asiático', 'ESPECIE', 3),
('Zea mays', 'Milho', 'ESPECIE', 4),
('Fabaceae', 'Leguminosas', 'FAMILIA', NULL),
('Phaseolus', 'Feijão', 'GENERO', 9),
('Phaseolus vulgaris', 'Feijão Comum', 'ESPECIE', 10);

-- Lotes de sementes
INSERT INTO LOTE_SEMENTES
(ID_Depositante, ID_Taxon, ID_Caixa, Data_Recebimento, Status) VALUES
(1, 5, 1, '2024-01-15', 'Armazenado'),
(1, 8, 1, '2024-01-15', 'Armazenado'),
(1, 11, 2, '2024-02-10', 'Armazenado'),
(2, 6, 3, '2024-02-20', 'Armazenado'),
(2, NULL, NULL, '2024-03-01', 'Em Quarentena'),
(4, 7, 4, '2024-03-05', 'Armazenado'),
(5, 5, 4, '2024-03-10', 'Armazenado'),
(3, NULL, NULL, '2024-03-15', 'Pendente');

-- Registros de auditoria
INSERT INTO MOVIMENTACAO_AUDITORIA
(ID_Lote, ID_Usuario, Tipo_Movimentacao, Data_Hora, Assinatura_Digital_Hash)
VALUES
(1, 1, 'Rececao', '2024-01-15 10:00:00',
encode(sha256('1Rececao2024-01-15 10:00:00'), 'hex')),
(1, 1, 'Armazenamento Final', '2024-01-16 14:00:00',
encode(sha256('1Armazenamento Final2024-01-16 14:00:00'), 'hex')),
(2, 1, 'Rececao', '2024-01-15 10:05:00',
encode(sha256('2Rececao2024-01-15 10:05:00'), 'hex')),
(5, 2, 'Rececao', '2024-03-01 09:00:00',
encode(sha256('5Rececao2024-03-01 09:00:00'), 'hex'));

```

2.3 Relatórios de Consulta

Relatório 1 – Inventário Geral de Lotes por Instituição e Localização

Este relatório apresenta uma visão completa do inventário, relacionando cada lote de sementes com sua instituição depositante, classificação taxonômica e localização física atual nas câmaras frias. É o relatório principal utilizado pelos administradores para auditorias e verificações de conformidade.

```
SELECT
    i.Nome_Instituicao          AS instituicao,
    i.Pais_Origem              AS pais,
    ct.Nome_Cientifico         AS especie,
    ct.Nome_Comum              AS nome_popular,
    ls.Status                   AS status_lote,
    ls.Data_Recebimento,
    c.Codigo_Rastreio_Caixa    AS caixa,
    loc.Sala_Camara,
    loc.Corredor,
    loc.Estante,
    loc.Prateleira
FROM LOTE_SEMENTES ls
JOIN INSTITUICAO i
    ON ls.ID_Depositante = i.ID_Depositante
LEFT JOIN CATEGORIA_TAXONOMICA ct
    ON ls.ID_Taxon = ct.ID_Taxon
LEFT JOIN CAIXA c
    ON ls.ID_Caixa = c.ID_Caixa
LEFT JOIN HISTORICO_ALOCACAO_CAIXA hac
    ON c.ID_Caixa = hac.ID_Caixa AND hac.Data_Saida IS NULL
LEFT JOIN LOCALIZACAO loc
    ON hac.ID_Localizacao = loc.ID_Localizacao
ORDER BY i.Pais_Origem, i.Nome_Instituicao, ct.Nome_Cientifico;
```

Relatório 2 – Histórico Completo de Movimentações de Auditoria

Este relatório lista todas as transações registradas na tabela de auditoria, identificando o lote afetado, o usuário responsável, o tipo da operação e o momento exato. É utilizado para auditorias internacionais de conformidade e rastreabilidade das amostras biológicas.

```
SELECT
    ma.ID_Movimentacao,
    ma.Tipo_Movimentacao,
    ma.Data_Hora,
    u.Nome                    AS responsavel,
    u.Tipo_Usuario,
    ls.ID_Lote,
    i.Nome_Instituicao,
    i.Pais_Origem,
    ct.Nome_Cientifico       AS especie,
    ls.Status                 AS status_atual,
    ma.Assinatura_Digital_Hash
FROM MOVIMENTACAO_AUDITORIA ma
JOIN USUARIO u
```

```

        ON ma.ID_Usuario = u.ID_Usuario
JOIN LOTE_SEMENTES ls
        ON ma.ID_Lote = ls.ID_Lote
JOIN INSTITUICAO i
        ON ls.ID_Depositante = i.ID_Depositante
LEFT JOIN CATEGORIA_TAXONOMICA ct
        ON ls.ID_Taxon = ct.ID_Taxon
ORDER BY ma.Data_Hora DESC;

```

Relatório 3 – Taxa de Ocupação das Caixas por Câmara

Este relatório calcula a ocupação atual de cada caixa e sua localização, exibindo a quantidade de lotes armazenados, os espaços disponíveis e o percentual de ocupação. Auxilia no planejamento logístico e na decisão de alocação de novos lotes.

```

SELECT
    loc.Sala_Camara,
    loc.Corredor,
    loc.Estante,
    loc.Prateleira,
    c.Codigo_Rastreio_Caixa,
    c.Capacidade_Maxima,
    COUNT(ls.ID_Lote) AS lotes_armazenados,
    c.Capacidade_Maxima - COUNT(ls.ID_Lote) AS espacos_disponiveis,
    ROUND(
        COUNT(ls.ID_Lote)::NUMERIC
        / c.Capacidade_Maxima * 100, 2
    ) AS percentual_ocupacao
FROM CAIXA c
JOIN HISTORICO_ALOCACAO_CAIXA hac
    ON c.ID_Caixa = hac.ID_Caixa AND hac.Data_Saida IS NULL
JOIN LOCALIZACAO loc
    ON hac.ID_Localizacao = loc.ID_Localizacao
LEFT JOIN LOTE_SEMENTES ls
    ON c.ID_Caixa = ls.ID_Caixa AND ls.Status = 'Armazenado'
GROUP BY
    loc.Sala_Camara, loc.Corredor, loc.Estante, loc.Prateleira,
    c.ID_Caixa, c.Codigo_Rastreio_Caixa, c.Capacidade_Maxima
ORDER BY loc.Sala_Camara, percentual_ocupacao DESC;

```

2.4 Triggers

RN02 – Controle de Capacidade Concorrente da Caixa

Este gatilho é executado antes de qualquer INSERT ou UPDATE na tabela LOTE_SEMENTES. Ele verifica se a caixa de destino já atingiu sua Capacidade_Maxima. Se sim, aborta a transação imediatamente com RAISE EXCEPTION, garantindo integridade física mesmo em cenários de inserções concorrentes de múltiplos operadores.

```
-- Função do gatilho de capacidade
CREATE OR REPLACE FUNCTION verificar_capacidade_caixa()
RETURNS TRIGGER AS $$
DECLARE
    v_lotes_atuais INT;
    v_cap_maxima    INT;
BEGIN
    -- Só valida se um ID_Caixa está sendo atribuído
    IF NEW.ID_Caixa IS NOT NULL THEN

        -- Conta os lotes já presentes na caixa (excluindo o próprio registro em
UPDATE)
        SELECT COUNT(*)
            INTO v_lotes_atuais
        FROM LOTE_SEMENTES
        WHERE ID_Caixa = NEW.ID_Caixa
            AND ID_Lote IS DISTINCT FROM NEW.ID_Lote;

        -- Obtém a capacidade máxima da caixa
        SELECT Capacidade_Maxima
            INTO v_cap_maxima
        FROM CAIXA
        WHERE ID_Caixa = NEW.ID_Caixa;

        -- Verifica o estouro
        IF v_lotes_atuais >= v_cap_maxima THEN
            RAISE EXCEPTION
                'Capacidade máxima da caixa % atingida. '
                'Limite: % lotes. Lotes atuais: %.',
                NEW.ID_Caixa, v_cap_maxima, v_lotes_atuais;
        END IF;

    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger associado
CREATE TRIGGER trg_verificar_capacidade_caixa
BEFORE INSERT OR UPDATE ON LOTE_SEMENTES
FOR EACH ROW
EXECUTE FUNCTION verificar_capacidade_caixa();
```

RN03 – Imutabilidade dos Registros Históricos (Bloqueio de DELETE)

Em conformidade com as diretrizes internacionais de auditoria da ONU, comandos DELETE são terminantemente bloqueados nas três tabelas protegidas. Três funções e três gatilhos são criados, um para cada tabela.

```
-- Bloqueio de DELETE em LOTE_SEMENTES
CREATE OR REPLACE FUNCTION bloquear_delete_lote()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION
        'Exclusão física proibida em LOTE_SEMENTES. '
        'Utilize UPDATE no campo Status para ''Inviavel'' ou ''Retirado''.';
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_bloquear_delete_lote
BEFORE DELETE ON LOTE_SEMENTES
FOR EACH ROW
EXECUTE FUNCTION bloquear_delete_lote();

-- Bloqueio de DELETE em HISTORICO_ALOCACAO_CAIXA
CREATE OR REPLACE FUNCTION bloquear_delete_historico()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION
        'Exclusão física proibida em HISTORICO_ALOCACAO_CAIXA. '
        'Registros de rastreamento logístico são imutáveis.';
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_bloquear_delete_historico
BEFORE DELETE ON HISTORICO_ALOCACAO_CAIXA
FOR EACH ROW
EXECUTE FUNCTION bloquear_delete_historico();

-- Bloqueio de DELETE em MOVIMENTACAO_AUDITORIA
CREATE OR REPLACE FUNCTION bloquear_delete_auditoria()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION
        'Exclusão física proibida em MOVIMENTACAO_AUDITORIA. '
        'Registros de auditoria são imutáveis por diretriz da ONU.';
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_bloquear_delete_auditoria
BEFORE DELETE ON MOVIMENTACAO_AUDITORIA
FOR EACH ROW
EXECUTE FUNCTION bloquear_delete_auditoria();
```

RN03 cont. – Registro Automático de Auditoria ao Alterar Status

Quando o campo Status de um lote é alterado para "Inviavel" ou "Retirado" via UPDATE, este gatilho dispara automaticamente um INSERT na tabela MOVIMENTACAO_AUDITORIA, registrando o responsável, a data/hora e gerando a assinatura digital hash. O ID do usuário atual é recuperado por meio de um parâmetro de sessão configurável.

```
-- Função de auditoria automática ao alterar Status
CREATE OR REPLACE FUNCTION registrar_auditoria_status()
RETURNS TRIGGER AS $$
DECLARE
    v_usuario_id    INT;
    v_tipo_mov      VARCHAR(30);
    v_hash_input    TEXT;
BEGIN
    -- Verifica se o Status mudou para um estado terminal
    IF NEW.Status IN ('Inviavel', 'Retirado')
        AND OLD.Status IS DISTINCT FROM NEW.Status THEN

        -- Recupera o usuário da sessão atual (configurado pelo backend)
        v_usuario_id := current_setting('app.current_user_id', true)::INT;

        -- Define o tipo de movimentação pelo novo status
        v_tipo_mov := CASE NEW.Status
            WHEN 'Inviavel' THEN 'Descarte Sanitario'
            WHEN 'Retirado' THEN 'Retirada'
        END;

        -- Gera a assinatura digital hash
        v_hash_input := NEW.ID_Lote::TEXT
            || v_tipo_mov
            || CURRENT_TIMESTAMP::TEXT;

        INSERT INTO MOVIMENTACAO_AUDITORIA (
            ID_Lote,
            ID_Usuario,
            Tipo_Movimentacao,
            Data_Hora,
            Assinatura_Digital_Hash
        ) VALUES (
            NEW.ID_Lote,
            v_usuario_id,
            v_tipo_mov,
            CURRENT_TIMESTAMP,
            encode(sha256(v_hash_input::bytea), 'hex')
        );

    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger associado (AFTER UPDATE, filtrando apenas mudanças de Status)
CREATE TRIGGER trg_auditoria_status_lote
AFTER UPDATE OF Status ON LOTE_SEMENTES
FOR EACH ROW
```

```
WHEN (OLD.Status IS DISTINCT FROM NEW.Status)
EXECUTE FUNCTION registrar_auditoria_status();
```

2.5 Tipos de Dados Não Convencionais

O sistema Svalbard VaultDB pode ser complementado com dois tipos de banco de dados não-convencionais para casos de uso específicos que extrapolam as capacidades do modelo relacional puro.

MongoDB (Banco de Dados Orientado a Documentos)

O MongoDB é ideal para armazenar fichas técnicas detalhadas de cada espécie, que possuem estrutura variável dependendo da categoria taxonômica. Uma espécie de gramínea pode ter dezenas de atributos agronômicos (resistência a pragas, período de floração, tolerância à seca) que não se aplicam a leguminosas. Em vez de criar centenas de colunas nullable na tabela CATEGORIA_TAXONOMICA, um documento MongoDB armazena essas características de forma flexível:

```
{
  "_id": "taxon_5",
  "nome_cientifico": "Triticum aestivum",
  "rank": "ESPECIE",
  "atributos_agronicos": {
    "ciclo_dias": 120,
    "resistencia_ferrugem": "alta",
    "tolerancia_seca": "media",
    "rendimento_ton_ha": 4.5,
    "proteina_percentual": 13.2
  },
  "origem_geografica": {
    "centro_diversidade": "Crescente Fértil",
    "paises_nativos": ["Irake", "Irã", "Turquia"]
  }
}
```

InfluxDB (Banco de Dados de Séries Temporais)

O InfluxDB é adequado para monitorar, em tempo real e com alta frequência, as condições ambientais das câmaras frias de Svalbard: temperatura, umidade e níveis de CO₂. Em vez de sobrecarregar o banco relacional com milhões de leituras de sensores IoT por dia, o InfluxDB armazena essas séries temporais de forma otimizada, permitindo alertas automáticos se a temperatura ultrapassar o limiar de -18°C e garantindo a rastreabilidade das condições de conservação.

3. PROJETO FÍSICO

3.1 Uso de Índices

Os índices abaixo foram definidos com base nas consultas mais frequentes do sistema: buscas por status de lotes, filtros por instituição e espécie, e consultas de localização de caixas. Índices em chaves estrangeiras aceleram os JOINS; índices em colunas de filtro aceleram as cláusulas WHERE.

```
-- Índices em chaves estrangeiras (aceleram JOINS)
CREATE INDEX idx_lote_depositante
    ON LOTE_SEMENTES (ID_Depositante);

CREATE INDEX idx_lote_taxon
    ON LOTE_SEMENTES (ID_Taxon);

CREATE INDEX idx_lote_caixa
    ON LOTE_SEMENTES (ID_Caixa);

CREATE INDEX idx_repr_depositante
    ON REPRESENTANTE (ID_Depositante);

CREATE INDEX idx_historico_caixa
    ON HISTORICO_ALOCACAO_CAIXA (ID_Caixa);

CREATE INDEX idx_historico_localizacao
    ON HISTORICO_ALOCACAO_CAIXA (ID_Localizacao);

CREATE INDEX idx_auditoria_lote
    ON MOVIMENTACAO_AUDITORIA (ID_Lote);

CREATE INDEX idx_auditoria_usuario
    ON MOVIMENTACAO_AUDITORIA (ID_Usuario);

CREATE INDEX idx_taxon_pai
    ON CATEGORIA_TAXONOMICA (ID_Pai_Taxon);

-- Índices em colunas de filtro frequente
CREATE INDEX idx_lote_status
    ON LOTE_SEMENTES (Status);

CREATE INDEX idx_historico_saida
    ON HISTORICO_ALOCACAO_CAIXA (Data_Saida)
    WHERE Data_Saida IS NULL;          -- índice parcial: apenas alocações ativas

CREATE INDEX idx_auditoria_data
    ON MOVIMENTACAO_AUDITORIA (Data_Hora DESC);

CREATE INDEX idx_taxon_rank
    ON CATEGORIA_TAXONOMICA (Rank_Taxonomico);

-- Índice composto para a política RLS (Caixa Preta)
CREATE INDEX idx_lote_depositante_status
    ON LOTE_SEMENTES (ID_Depositante, Status);
```

3.2 Uso de Tablespaces

A separação dos dados em tablespaces distintos permite que tabelas de acesso frequente e seus índices sejam armazenados em discos de alta performance (SSD NVMe), enquanto tabelas de log e histórico, que são gravadas sequencialmente mas raramente consultadas, residem em volumes de maior capacidade.

```
-- Criação dos tablespaces
CREATE TABLESPACE ts_dados_principais
  LOCATION '/mnt/nvme/tablespaces/dados';

CREATE TABLESPACE ts_indices
  LOCATION '/mnt/nvme/tablespaces/indices';

CREATE TABLESPACE ts_historico
  LOCATION '/mnt/hdd/tablespaces/historico';

-- Movendo tabelas principais para SSD
ALTER TABLE LOTE_SEMENTES
  SET TABLESPACE ts_dados_principais;

ALTER TABLE CATEGORIA_TAXONOMICA
  SET TABLESPACE ts_dados_principais;

ALTER TABLE INSTITUICAO
  SET TABLESPACE ts_dados_principais;

-- Movendo tabelas de log para HDD (alta capacidade)
ALTER TABLE MOVIMENTACAO_AUDITORIA
  SET TABLESPACE ts_historico;

ALTER TABLE HISTORICO_ALOCACAO_CAIXA
  SET TABLESPACE ts_historico;

-- Movendo índices críticos para SSD
ALTER INDEX idx_lote_depositante_status
  SET TABLESPACE ts_indices;

ALTER INDEX idx_lote_status
  SET TABLESPACE ts_indices;

ALTER INDEX idx_historico_saida
  SET TABLESPACE ts_indices;

ALTER INDEX idx_auditoria_data
  SET TABLESPACE ts_indices;
```

3.3 Otimização dos Relatórios de Consulta

A otimização é realizada utilizando EXPLAIN ANALYZE para identificar gargalos e reescrever as consultas aproveitando os índices criados na seção 3.1.

```
-- Relatório 1 otimizado com EXPLAIN ANALYZE
EXPLAIN ANALYZE
SELECT
```

```

        i.Nome_Instituicao, i.Pais_Origem,
        ct.Nome_Cientifico, ct.Nome_Comum,
        ls.Status, ls.Data_Recebimento,
        c.Codigo_Rastreio_Caixa,
        loc.Sala_Camara, loc.Corredor, loc.Estante, loc.Prateleira
FROM LOTE_SEMENTES ls
JOIN INSTITUICAO i
    ON ls.ID_Depositante = i.ID_Depositante -- usa idx_lote_depositante
LEFT JOIN CATEGORIA_TAXONOMICA ct
    ON ls.ID_Taxon = ct.ID_Taxon -- usa idx_lote_taxon
LEFT JOIN CAIXA c
    ON ls.ID_Caixa = c.ID_Caixa -- usa idx_lote_caixa
LEFT JOIN HISTORICO_ALOCACAO_CAIXA hac
    ON c.ID_Caixa = hac.ID_Caixa
    AND hac.Data_Saida IS NULL -- usa idx_historico_saida (parcial)
LEFT JOIN LOCALIZACAO loc
    ON hac.ID_Localizacao = loc.ID_Localizacao
ORDER BY i.Pais_Origem, i.Nome_Instituicao, ct.Nome_Cientifico;

```

-- Relatório 2 otimizado

EXPLAIN ANALYZE

```

SELECT
    ma.ID_Movimentacao, ma.Tipo_Movimentacao, ma.Data_Hora,
    u.Nome, u.Tipo_Usuario,
    ls.ID_Lote, i.Nome_Instituicao, ct.Nome_Cientifico, ls.Status
FROM MOVIMENTACAO_AUDITORIA ma -- usa idx_auditoria_data para ORDER BY
JOIN USUARIO u ON ma.ID_Usuario = u.ID_Usuario
JOIN LOTE_SEMENTES ls ON ma.ID_Lote = ls.ID_Lote -- usa idx_auditoria_lote
JOIN INSTITUICAO i ON ls.ID_Depositante = i.ID_Depositante
LEFT JOIN CATEGORIA_TAXONOMICA ct ON ls.ID_Taxon = ct.ID_Taxon
ORDER BY ma.Data_Hora DESC;

```

-- Relatório 3 otimizado

EXPLAIN ANALYZE

```

SELECT
    loc.Sala_Camara, loc.Corredor, loc.Estante, loc.Prateleira,
    c.Codigo_Rastreio_Caixa, c.Capacidade_Maxima,
    COUNT(ls.ID_Lote) AS lotes_armazenados,
    c.Capacidade_Maxima - COUNT(ls.ID_Lote) AS espacos_disponiveis,
    ROUND(COUNT(ls.ID_Lote)::NUMERIC
        / c.Capacidade_Maxima * 100, 2) AS percentual_ocupacao
FROM CAIXA c
JOIN HISTORICO_ALOCACAO_CAIXA hac -- usa idx_historico_saida (parcial)
    ON c.ID_Caixa = hac.ID_Caixa AND hac.Data_Saida IS NULL
JOIN LOCALIZACAO loc
    ON hac.ID_Localizacao = loc.ID_Localizacao
LEFT JOIN LOTE_SEMENTES ls -- usa idx_lote_depositante_status
    ON c.ID_Caixa = ls.ID_Caixa AND ls.Status = 'Armazenado'
GROUP BY loc.Sala_Camara, loc.Corredor, loc.Estante, loc.Prateleira,
        c.ID_Caixa, c.Codigo_Rastreio_Caixa, c.Capacidade_Maxima
ORDER BY loc.Sala_Camara, percentual_ocupacao DESC;

```

3.4 Criação de Usuários e Privilégios

São definidos três perfis de acesso: o administrador da instalação, o representante institucional e o analista de auditoria externa (ONU), cada um com privilégios rigorosamente mínimos.

```
-- Criação das roles
CREATE ROLE role_administrador;
CREATE ROLE role_representante;
CREATE ROLE role_auditor_onu;

-- Privilégios do Administrador (acesso total)
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public
    TO role_administrador;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public
    TO role_administrador;

-- Privilégios do Representante (acesso restrito ao seu próprio acervo via RLS)
GRANT SELECT, INSERT ON LOTE_SEMENTES
    TO role_representante;
GRANT SELECT ON CATEGORIA_TAXONOMICA, INSTITUICAO, CAIXA
    TO role_representante;
GRANT SELECT ON MOVIMENTACAO_AUDITORIA
    TO role_representante;

-- Privilégios do Auditor da ONU (somente leitura)
GRANT SELECT ON ALL TABLES IN SCHEMA public
    TO role_auditor_onu;

-- Criação de usuários concretos
CREATE USER admin_erik WITH PASSWORD 'S@ltnvault#2024!Erik';
CREATE USER repr_carlos WITH PASSWORD 'S@ltnvault#2024!Carlos';
CREATE USER auditor_onu_001 WITH PASSWORD 'S@ltnvault#2024!ONU01';

GRANT role_administrador TO admin_erik;
GRANT role_representante TO repr_carlos;
GRANT role_auditor_onu TO auditor_onu_001;
```

- **Perfis definidos** – Administrador (role_administrador): Possui ALL PRIVILEGES em todas as tabelas e sequências. É o único perfil autorizado a realizar alterações estruturais (ALTER TABLE), criar novos índices e gerenciar backups. Utilizado pelos operadores locais de Svalbard e pelos DBAs.
- Representante (role_representante): Possui SELECT e INSERT em LOTE_SEMENTES, mas o RLS (seção 3.5) garante que só visualize os lotes de sua própria instituição. Pode consultar dados taxonômicos e de localização para suporte científico.
- Auditor da ONU (role_auditor_onu): Acesso somente leitura (SELECT) em todas as tabelas, permitindo auditorias internacionais sem risco de alteração de dados.

3.5 Definição de RLS para Usuários

A Regra de Negócio RN01 – "Caixa Preta" – é implementada via Row-Level Security (RLS) nativa do PostgreSQL. Quando um representante realiza uma consulta em LOTE_SEMENTES, o banco de dados injeta automaticamente uma cláusula WHERE que filtra apenas os lotes pertencentes à sua instituição, tornando-o completamente cego para os dados de outros países.

```
-- Habilitar RLS nas tabelas sensíveis
ALTER TABLE LOTE_SEMENTES ENABLE ROW LEVEL SECURITY;
ALTER TABLE MOVIMENTACAO_AUDITORIA ENABLE ROW LEVEL SECURITY;

-- — Política "Caixa Preta" para LOTE_SEMENTES —————
-- O representante só enxerga lotes da sua própria instituição
CREATE POLICY politica_caixa_preta_select
ON LOTE_SEMENTES
FOR SELECT
TO role_representante
USING (
    ID_Depositante = (
        SELECT r.ID_Depositante
        FROM REPRESENTANTE r
        JOIN USUARIO u ON r.ID_Usuario = u.ID_Usuario
        WHERE u.Login = current_user
    )
);

-- O representante só pode inserir lotes na sua própria instituição
CREATE POLICY politica_caixa_preta_insert
ON LOTE_SEMENTES
FOR INSERT
TO role_representante
WITH CHECK (
    ID_Depositante = (
        SELECT r.ID_Depositante
        FROM REPRESENTANTE r
        JOIN USUARIO u ON r.ID_Usuario = u.ID_Usuario
        WHERE u.Login = current_user
    )
);

-- — Política de auditoria para MOVIMENTACAO_AUDITORIA —————
-- O representante só visualiza auditorias de seus próprios lotes
CREATE POLICY politica_auditoria_representante
ON MOVIMENTACAO_AUDITORIA
FOR SELECT
TO role_representante
USING (
    ID_Lote IN (
        SELECT ls.ID_Lote
        FROM LOTE_SEMENTES ls
        JOIN REPRESENTANTE r ON ls.ID_Depositante = r.ID_Depositante
        JOIN USUARIO u ON r.ID_Usuario = u.ID_Usuario
        WHERE u.Login = current_user
    )
);
```

```
-- Administradores e auditores da ONU bypass RLS (acesso irrestrito)
ALTER TABLE LOTE_SEMENTES
    FORCE ROW LEVEL SECURITY;
ALTER ROLE role_administrador BYPASSRLS;
ALTER ROLE role_auditor_onu    BYPASSRLS;
```

3.6 Comando para Backup Lógico

O backup lógico exporta as definições de tabelas, triggers, dados e constraints para um arquivo SQL legível por humanos, adequado para migração entre versões do PostgreSQL e para auditorias de conformidade.

```
pg_dump \
    -U postgres \
    -d svalbard_vaultdb \
    -F p \
    -v \
    --no-owner \
    --no-acl \
    -f /backups/logico/svalbard_$(date +%Y%m%d_%H%M).sql

-- Parâmetros explicados:
-- -F p    : formato plain (SQL legível)
-- -v      : verbose (exibe progresso)
-- --no-owner : omite comandos ALTER OWNER (portabilidade)
-- --no-acl  : omite GRANT/REVOKE
```

3.7 Comandos para Backup Físico

O backup físico é realizado com `pg_basebackup`, copiando os arquivos binários do cluster PostgreSQL. O parâmetro `-X stream` é essencial para um sistema crítico como o Svalbard VaultDB, pois inclui os WAL (Write-Ahead Logs) gerados durante a janela de backup, garantindo que nenhuma transação seja perdida mesmo que o backup leve horas para ser concluído.

```
pg_basebackup \
    -h localhost \
    -D /backups/fisico/svalbard_$(date +%Y%m%d) \
    -U replicator \
    -P \
    -v \
    -Ft \
    -z \
    -X stream

-- Parâmetros explicados:
-- -D      : diretório de destino do backup
-- -Ft     : formato tar (compactável)
-- -z      : compressão gzip
-- -X stream : inclui WAL via streaming (nenhuma transação perdida)
-- -P      : exibe progresso percentual
```


4. MODELO NOSQL

Para complementar o Svalbard VaultDB, exploramos a integração com um Banco de Dados Orientado a Grafos utilizando o Neo4j. Embora o modelo relacional seja ideal para garantir integridade, auditoria e conformidade internacional, o Neo4j supera o PostgreSQL em cenários que envolvem o mapeamento de relações filogenéticas profundas e análises de diversidade genética entre espécies.

Componentes do Modelo em Grafo

- Nós (Nodes): Representam entidades do domínio botânico e institucional — Espécie, Gênero, Família, Instituição, País, Lote.
- Arestas (Relationships): Representam conexões tipadas e direcionadas — PERTENCE_A (espécie → gênero), DEPOSITOU (instituição → lote), ORIGINADO_DE (lote → país), CRUZA_COM (espécie ↔ espécie para parentes genéticos).
- Propriedades: Cada nó e aresta pode conter atributos (ex.: distância genética em CRUZA_COM, quantidade de lotes em DEPOSITOU).

Aplicações Práticas no Svalbard VaultDB

1. **Árvore Filogenética Navegável:** Ao contrário do autorrelacionamento recursivo da tabela `CATEGORIA_TAXONOMICA`, que exige consultas `WITH RECURSIVE` para navegar na hierarquia, o Neo4j permite traversals de profundidade arbitrária em $O(1)$ por hop. A consulta para encontrar todos os parentes genéticos de *Triticum aestivum* dentro de 3 graus de separação seria:

```
MATCH (esp:Especie {nome: 'Triticum aestivum'})-[:CRUZA_COM*1..3]-(parente)
RETURN parente.nome, parente.rank, length(path) AS grau_parentesco
ORDER BY grau_parentesco;
```

2. **Análise de Diversidade por País:** Identificar quais países possuem a maior diversidade genética depositada, considerando não apenas a contagem de lotes mas a distância filogenética entre as espécies depositadas:

```
MATCH (p:País)-[:ORIGINADO_DE]-(i:Instituicao)-[:DEPOSITOU]-(l:Lote)
      -[:CLASSIFICADO_COMO]-(e:Especie)-[:PERTENCE_A*]-(f:Familia)
RETURN p.nome AS pais,
       COUNT(DISTINCT e) AS especies_unicas,
       COUNT(DISTINCT f) AS familias_distintas,
       COUNT(l) AS total_lotes
ORDER BY especies_unicas DESC;
```

Conclusão: A utilização do Neo4j como banco de dados complementar ao PostgreSQL no Svalbard VaultDB permite explorar a riqueza das relações biológicas entre as espécies conservadas, oferecendo ferramentas poderosas para pesquisadores que estudam diversidade genética e estratégias de conservação ex situ. O modelo relacional permanece como a fonte

primária de verdade para dados operacionais, enquanto o grafo serve como camada analítica e de descoberta científica.